

Lab 11

Textures in OpenGL

Objectives

- We will learn to implement texture in OpenGL.

Programming Exercises

(90 minutes)

In this lab, we will learn how to implement texturing in OpenGL. An overview of the tasks in this lab is as follows:

Task 1: Adding libraries to load images

Task 2: Setting up textures in OpenGL

Task 3: Adding textures into objects

Task 4: Practice and experiment

Task 1: Adding Libraries to Load Images (15 minutes)

Task 1A: Copy the required library files

In order for us to be able to load images to be used for texturing, we will need to rely on 3rd party library to do so since OpenGL does not provide any platform-dependant library for such purposes.

In this lab, 3rd party libraries are provided to you with these instructions in the form of a set of files which is needed for this lab, the files are:

- `jpeg62.dll` and `libjpeg.dll.a` (library file for handling and loading jpeg images)
- Sub-folder `CGLab11Utils`, which contains `CGimageloader.hpp`, `CGimageloader.cpp`, `jmorecfg.h`, `jconfig.h` and `jpeglib.h` (all for loading image files)
- Sub-folder `images` (which contains `marble1.jpg`, `wood1.jpg`, `brick1.jpg`, `rocks.bmp` and other sample *bmp* or *jpg* image files)

Please copy the `jpeg62.dll` and `libjpeg.dll.a` files in the same folder as the executable file to be created in this lab (typically it would be the folder where your project file is). Copy the whole sub-folders `CGLab11Utils` and sub-folder `images` into the same folder as well.

This 3rd party library is free but please read the comments in the program regarding terms of use and copyrights.

Please take note that these files are for Windows environment only. For other platform such as GNU/Linux, you will need to find alternative libraries to load images for that particular platform.

Task 1B: Add libraries into the project

Do the following steps:

- a. Add `libjpeg.dll.a` into the project: From Code::Blocks IDE, right-click at the project file and select 'Build options', then click at the 'Linker' tab and add `libjpeg.dll.a` into the project
- b. Click at the 'Directories' tab and then click at the 'Compiler' tab under the 'Directories' tab, then add the `CGLab11Utils` sub-folder there.

c. Add the `CGimageloader.cpp` program into your project.

By now, you should know how to create a project file and add files into it.

Task 2: Setting up textures in OpenGL (15 minutes)

To set up texture, modify the `MyVirtualWorld` class in `CGLab10and11.hpp` file as highlighted below:

```
#include <GL/glut.h>
#include "CGLab10and11.hpp"

#include "CGimageloader.hpp"

using namespace CGLab10and11;

class MyVirtualWorld
{
public:
    ...
    ...
    void setupTextures();
    void toggleTextures();
private:
    ...
    ...
    int numberOfTexture;
    GLuint* textureNumbers;
    bool textureison;
};
```

Modify the `CGLab10and11.cpp` file accordingly as below:

```
MyVirtualWorld::MyVirtualWorld()
{
    ...
    ...
    numberOfTexture = 0;
    textureNumbers = NULL;
    textureison = false;
}

MyVirtualWorld::~MyVirtualWorld()
{
    if (textureNumbers != NULL)
        delete[] textureNumbers;
}

void MyVirtualWorld::init()
{
    setupLights();
    setupTextures();
    timeold = glutGet(GLUT_ELAPSED_TIME);
}

void MyVirtualWorld::setupTextures()
{

```

```

numberOfTexture = 3;
textureNumbers = new GLuint[numberOfTexture];

//Set pixel storage mode
glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

//set texture environment parameters
glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);

//Generate texture map identifiers
glGenTextures(numberOfTexture, textureNumbers);

//Read image data into memory
MyImage myimage[numberOfTexture];
myimage[0].loadJPG("images/marble1.jpg");
myimage[1].loadJPG("images/wood1.jpg");
myimage[2].loadJPG("images/brick1.jpg");
//Try this: myimage[2].loadBMP("images/rocks.bmp");

//for each texture map identifiers
for (int i=0; i<numberOfTexture; i++)
{
    //Bind a named texture to a texturing target
    glBindTexture(GL_TEXTURE_2D, textureNumbers[i]);

    //Set texture parameters
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);

    //Set texture map
    glTexImage2D(GL_TEXTURE_2D,                // defining a 2D texture
                0,                             // level (not multi-resolution)
                GL_RGBA,                       // format of OpenGL's texture
                myimage[i].width, myimage[i].height, // dimensions of the image
                0,                             // border
                GL_RGB, GL_UNSIGNED_BYTE,      // format of image
                myimage[i].buffer              // pixel data of image
    );
}

textureison = true;
}

void MyVirtualWorld::toggleTextures()
{
    textureison = !textureison;
}

```

Please refer to the OpenGL “Bluebook” to find out what each of the functions calls in `MyVirtualWorld::setupTextures()` mean.

Modify the `CGLab10and11main.cpp` file as highlighted below to bind the function key F4 for toggling textures:

```

void mySpecialFunc(int key, int x, int y)
{
    switch (key)
    {

```



```

//The size of each quad used is:
//      quadSizeX x quadSizeZ
GLfloat quadSizeX = 0.25f;
GLfloat quadSizeZ = 0.25f;

//For each quad, the 2 end corners have coord below:
//      (x1, 0.0, z1) and (x2, 0.0, z2)
// and the texture coord for those corners are:
//      (s=s1, t=t1) and (s=s2, t=t2)
GLfloat x1, z1, x2, z2;
GLfloat s1, t1, s2, t2;

//Given a vertex (x,y,z) on the roof, the texture coord (s,t) is given by:
//      s = (texSMax/xdim) * (x-xmin)
//      t = (texTMax/zdim) * (z-zmin)
//Thus, the pre-calculated scaling factors as below:
GLfloat texScaleX = texSMax/xdim;
GLfloat texScaleZ = texTMax/zdim;

if (textureison)
{
    glEnable(GL_TEXTURE_2D);
    glBindTexture(GL_TEXTURE_2D, textureNumbers[0]);
}

glPushMatrix();
for (z1=zmin; z1<zmax; z1+=quadSizeZ)
{
    z2 = z1 + quadSizeZ;
    t1 = texScaleZ*(z1-zmin);
    t2 = texScaleZ*(z2-zmin);
    for (x1=xmin; x1<xmax; x1+=quadSizeX)
    {
        x2 = x1 + quadSizeX;
        s1 = texScaleX*(x1-xmin);
        s2 = texScaleX*(x2-xmin);

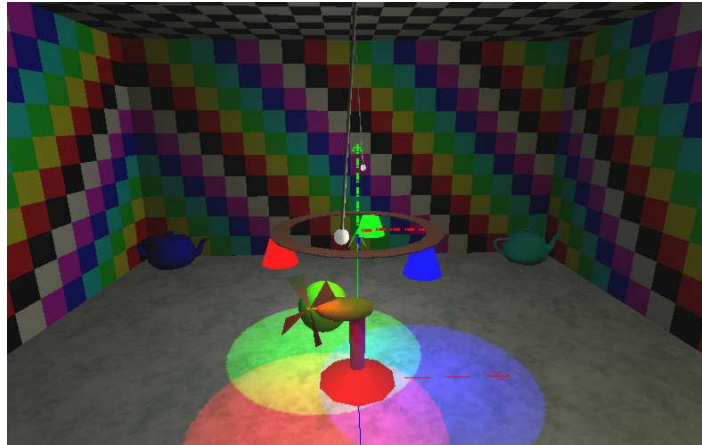
        glColor3fv( mywhite );
        glBegin(GL_QUADS);
            //pointing upwards
            glNormal3f(0.0f, 1.0f, 0.0f);

            //Note: Anti-clockwise as looking from inside
            glTexCoord2f(s1, t1);
            glVertex3f(x1, 0.0f, z1);
            glTexCoord2f(s1, t2);
            glVertex3f(x1, 0.0f, z2);
            glTexCoord2f(s2, t2);
            glVertex3f(x2, 0.0f, z2);
            glTexCoord2f(s2, t1);
            glVertex3f(x2, 0.0f, z1);

        glEnd();
    }
}
glPopMatrix();
glDisable(GL_TEXTURE_2D);
}

```

Run the program and you should see the output below:



Try the following:

- Press **F1** to see the world in wire-frame mode, take note how tiny are the quads and how many they are. We use very small quads here in order to have good lighting effect.
- Press **F3** to turn on or off lighting mode, you still will be able to see the textures but without lighting effect if lighting mode is turn off.
- Press **F4** to turn on or off texture mode.

Task 3B: Adding texture to roof

We will put texture on the roof now, recall in Lab 10 that the roof has been drawn with chequered boxes.

Replace the **whole** `MyVirtualWorld::drawRoof()` function in `CGLab10and11.cpp` file in Lab 10 with the improved (more efficient and neater code, but doing the same thing as in Lab 10) version below. Code related to textures has been highlighted for you.

```
void MyVirtualWorld::drawRoof()
{
    //Coordinates for the roof is:
    //      xmin<=x<=xmax,  y=height,  zmin<=z<=zmax
    //      (declared in constructor of MyVirtualWorld)

    //Texture coordinates for vertex (xmin, height, zmin) is:
    //      (s=0, t=0)
    //Texture coordinates for vertex (xmax, height, zmax) is:
    //      (s=texSMax, t=texTMax)
    GLfloat texSMax = 10.0f;
    GLfloat texTMax = 10.0f;

    //The roof consists of checker boxes, dimension of each checker box:
    //      checkboxSizeX x checkboxSizeZ
    GLfloat checkboxSizeX = 5.0f;
    GLfloat checkboxSizeZ = 5.0f;

    //Number of quads per checker box:
```

```

//      quadsPerCheckerboxX x quadsPerCheckerboxZ
//      (for each directions)
GLint quadsPerCheckerboxX = 2;
GLint quadsPerCheckerboxZ = 2;

//Thus the size of each quad is:
//      quadSizeX x quadSizeZ
GLfloat quadSizeX = checkBoxSizeX / quadsPerCheckerboxX;
GLfloat quadSizeZ = checkBoxSizeZ / quadsPerCheckerboxZ;

//Thus, for each quad, the 2 end corners have coord below:
//      (x1, height, z1) and (x2, height, z2)
// and the texture coord for those corners are:
//      (s=s1, t=t1) and (s=s2, t=t2)
GLfloat x1, z1, x2, z2;
GLfloat s1, t1, s2, t2;

//Given a vertex (x,y,z) on the roof, the texture coord (s,t) is given by:
//      s = (texSMax/xdim) * (x-xmin)
//      t = (texTMax/zdim) * (z-zmin)
//Thus, the pre-calulated scaling factors as below:
GLfloat texScaleX = texSMax/xdim;
GLfloat texScaleZ = texTMax/zdim;

static GLfloat* color[] = { myblack, mywhite };
//take note, the mywhite and myblack etc. are defined in CGLabmain.hpp

if (textureison)
{
    glEnable(GL_TEXTURE_2D);
    glBindTexture(GL_TEXTURE_2D, textureNumbers[1]);
}

glPushMatrix();

int startcolorno = 0;
for (GLfloat z=zmin; z<zmax; z+=checkBoxSizeZ)
{
    int colorno = startcolorno;
    for (GLfloat x=xmin; x<xmax; x+=checkBoxSizeX)
    {
        GLint i,j;
        //draw all the quads in the checker box
        glColor3fv( color[colorno] );
        for (j=0, z1 = z, z2 = z+quadSizeZ;
            j<quadsPerCheckerboxZ;
            ++j, z1+=quadSizeZ, z2+=quadSizeZ)
        {
            t1 = texScaleZ*(z1-zmin);
            t2 = texScaleZ*(z2-zmin);
            for (i=0, x1 = x, x2 = x+quadSizeX;
                i<quadsPerCheckerboxX;
                ++i, x1+=quadSizeX, x2+=quadSizeX)
            {
                s1 = texScaleX*(x1-xmin);
                s2 = texScaleX*(x2-xmin);
                glBegin(GL_QUADS);
                //pointing downwards
                glNormal3f(0.0f, -1.0f, 0.0f);

```

```

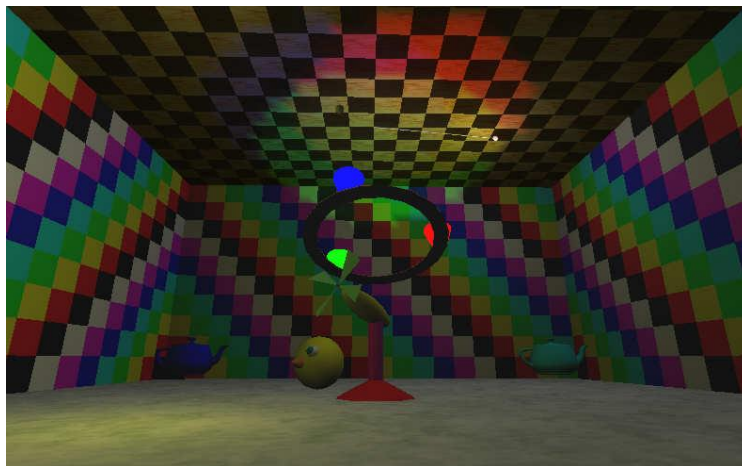
        //Note: Anti-clockwise as looking from inside
        glTexCoord2f(s1, t1);
        glVertex3f(x1, height, z1);
        glTexCoord2f(s2, t1);
        glVertex3f(x2, height, z1);
        glTexCoord2f(s2, t2);
        glVertex3f(x2, height, z2);
        glTexCoord2f(s1, t2);
        glVertex3f(x1, height, z2);
        glEnd();
    } //for i
} //for j

    if (colorno==0) colorno=1;
    else colorno = 0;
} //for x
if (startcolorno==0) startcolorno=1;
else startcolorno = 0;
} //for z

glPopMatrix();
glDisable(GL_TEXTURE_2D);
}

```

Run the program and you should see the output below:



Try using other images to do the texturing. Take note that images that are to be used for textures in this lab must be of **two dimensions**.

Task 4: Practice and Experiment (35 minutes)

Now your task is to add textures to the 4 walls.

Replace the **whole** `MyVirtualWorld::drawWalls()` function in `CGLab10and11.cpp` file in Lab 10 with the improved (more efficient and neater code, but doing the same thing as in Lab 10) version below. Code related to textures has been highlighted for you.


```

void MyVirtualWorld::drawWalls()
{
    //Note: the method of building the walls is not exactly the best way
    //      to do it, the better way is to build one wall and transform the same
    //      wall to another location to build another
    static GLfloat* color[] =
        { myblack, myred, myyellow, mygreen,
          mycyan, myblue, mymagenta, mywhite };
    //take note, the myred, myblue etc. are defined in CGLabmain.hpp

    if (textureison)
    {
        glEnable(GL_TEXTURE_2D);
        glBindTexture(GL_TEXTURE_2D, textureNumbers[2]);
    }

    GLfloat texSMax = 5.0f;
    GLfloat texTMax = 5.0f;

    GLfloat brickSizeY = 5.0f;
    GLfloat brickSizeZ = 5.0f;

    GLint quadsPerBrickY = 2;
    GLint quadsPerBrickZ = 2;

    GLfloat quadSizeY = brickSizeY / quadsPerBrickY;
    GLfloat quadSizeZ = brickSizeZ / quadsPerBrickZ;

    GLfloat y1, z1, y2, z2;
    GLfloat s1, t1, s2, t2;

    GLfloat texScaleY = texSMax/ydim;
    GLfloat texScaleZ = texTMax/zdim;

    glPushMatrix();

    int startcolorno = 0;
    for (GLfloat z=zmin; z<zmax; z+=brickSizeZ)
    {
        int colorno = startcolorno;
        for (GLfloat y=0; y<height; y+=brickSizeY)
        {
            GLint i,j;
            //draw all the quads in the checker box
            if (textureison) //use single color if texture is used
                colorno = 7;
            glColor3fv( color[colorno] );
            for (j=0, z1 = z, z2 = z+quadSizeZ;
                 j<quadsPerBrickZ;
                 ++j, z1+=quadSizeZ, z2+=quadSizeZ)
            {
                t1 = texScaleZ*(z1-zmin);
                t2 = texScaleZ*(z2-zmin);
                for (i=0, y1 = y, y2 = y+quadSizeY;
                     i<quadsPerBrickY;
                     ++i, y1+=quadSizeY, y2+=quadSizeY)
                {
                    s1 = texScaleY*y1; //ymin is 0.0
                    s2 = texScaleY*y2; //ymin is 0.0
                    glBegin(GL_QUADS);

```

```

        //Wall on positive-x, normal pointing to negative-x
        glNormal3f(-1.0f, 0.0f, 0.0f);
        //Note: Anti-clockwise as looking from inside
        glTexCoord2f(s1, t1);
        glVertex3f(xmax, y1, z1);
        glTexCoord2f(s1, t2);
        glVertex3f(xmax, y1, z2);
        glTexCoord2f(s2, t2);
        glVertex3f(xmax, y2, z2);
        glTexCoord2f(s2, t1);
        glVertex3f(xmax, y2, z1);

        //Wall on negative-x, normal pointing to positive-x
        glNormal3f(1.0f, 0.0f, 0.0f);
        //Note: Anti-clockwise as looking from inside
        glVertex3f(xmin, y1, z1);
        glVertex3f(xmin, y2, z1);
        glVertex3f(xmin, y2, z2);
        glVertex3f(xmin, y1, z2);

        glEnd();
    } //for i
} //for j

++colorno;
if (colorno>7) colorno=0;
} //for y
++startcolorno;
if (startcolorno>7) startcolorno=0;
} //for z

GLfloat brickSizeX = 5.0f;
GLint quadsPerBrickX = 2;
GLfloat quadSizeX = brickSizeX / quadsPerBrickX;
GLfloat x1, x2;

glPushMatrix();

startcolorno = 0;
for (GLfloat x=xmin; x<xmax; x+=brickSizeX)
{
    int colorno = startcolorno;
    for (GLfloat y=0; y<height; y+=brickSizeY)
    {

        GLint i,j;
        //draw all the quads in the checker box
        if (textureison) //use single color if texture is used
            colorno = 7;
            glColor3fv( color[colorno] );
        for (j=0, x1 = x, x2 = x+quadSizeX;
            j<quadsPerBrickX;
            ++j, x1+=quadSizeX, x2+=quadSizeX)
        {
            for (i=0, y1 = y, y2 = y+quadSizeY;
                i<quadsPerBrickY;
                ++i, y1+=quadSizeY, y2+=quadSizeY)
            {
                glBegin(GL_QUADS);
                //Wall on positive-z, normal pointing to negative-z
                glNormal3f(0.0f, 0.0f, -1.0f);

```

```

        //Note: Anti-clockwise as looking from inside
        glVertex3f(x1, y1, zmax);
        glVertex3f(x1, y2, zmax);
        glVertex3f(x2, y2, zmax);
        glVertex3f(x2, y1, zmax);

        //Wall on negative-z, normal pointing to positive-z
        glNormal3f(0.0f, 0.0f, 1.0f);
        //Note: Anti-clockwise as looking from inside
        glVertex3f(x1, y1, zmin);
        glVertex3f(x2, y1, zmin);
        glVertex3f(x2, y2, zmin);
        glVertex3f(x1, y2, zmin);

        glEnd();
    } //for i
} //for j

++colorno;
if (colorno>7) colorno=0;
} //for y
++startcolorno;
if (startcolorno>7) startcolorno=0;
} //for z

glPopMatrix();
glDisable(GL_TEXTURE_2D);
}

```

If you run the program, you will notice that only one wall has been textured so far. For the other three walls, they are either not textured at all or they are textured incorrectly (**Note: Find out why!**).

Try pressing F3 or F4 to see the world with either lighting turn off or textures turn off. Take note that the color pattern which we created (when texture is off) will not be used if texture is on.

Your task now is to complete the rest of the textures on the walls. Your output should look something like this in the end:



HAVE FUN, LET THERE BE TEXTURES!!

THIS IS THE **LAST LAB**. HOPE YOU HAVE BEEN HAVING FUN IN ALL YOUR LABS!!

Extra Exercises

Take note that the textures that we have experimented in this lab are all texture mapping done on a flat surface. It is possible that textures be added onto curve surfaces as well. You may also want to try “wrapping” a texture onto a quadric object such as a sphere, cylinder or disc to produce very nice effects. Try to find out how to do them so that you can produce more realistic objects.

There are many more advanced features that you can actually implement in OpenGL, but it has not been possible to cover them in this very basic and introductory lab course on OpenGL due to time constraints. So, please do explore them on their own if you are really interested in Computer Graphics programming. There are lots of materials on the Internet and books in the library that you can read. And remember, PRACTICE makes PERFECT!

* END OF LAB *